



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

Tópicos Avanzados en Teoría de la Computación - IIC3810

Tarea 2

Fecha de entrega: jueves 8 de octubre

Preguntas

1. [2 puntos] En esta pregunta usted deberá implementar en Python una librería que permita utilizar el algoritmo de Deutsch-Jozsa. Esta librería debe estar en el archivo `deutsch_jozsa.py`.

Recuerde que un oráculo para una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ es un circuito cuántico que implementa una transformación unitaria U_f tal que para todo $x \in \{0, 1\}^n$ y todo $y \in \{0, 1\}$:

$$U_f |y 0^r x\rangle = |(y \oplus f(x)) 0^r x\rangle,$$

donde $r \geq 0$ es el número de qubits auxiliares utilizados por el oráculo. Vale decir, un oráculo para f tiene $n + r + 1$ qubits, y debe dejar los qubits auxiliares en el estado $|0^r\rangle$. Siguiendo la convención de Qiskit, el qubit 0 corresponde al símbolo de más a la derecha en $|y 0^r x\rangle$. Así, x se almacena en los qubits $0, \dots, n - 1$, los qubits auxiliares son los qubits $n, \dots, n + r - 1$, y el valor y se almacena en el qubit $n + r$. Además, para cada $i \in \{0, \dots, n - 1\}$ denotamos por x_i el símbolo de x que se almacena en el qubit i , vale decir, $x = x_{n-1} \dots x_1 x_0$.

En esta tarea, un oráculo es un objeto de la clase `QuantumCircuit` de Qiskit que implementa U_f . Note que a partir de un oráculo no es posible deducir el valor de n : el atributo `num_qubits` de `QuantumCircuit` entrega el valor $n + r + 1$, pero no indica cuál es el valor de r . Por esta razón, las funciones que usted debe implementar reciben como parámetros un oráculo y el número n de bits de entrada de f . En particular, su librería debe implementar las siguientes funciones:

- Una función `circuito_deutsch_jozsa` con el siguiente encabezado:

```
def circuito_deutsch_jozsa(oraculo: QuantumCircuit, n: int) -> QuantumCircuit:
```

Esta función recibe como parámetros un oráculo `oraculo` para una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ y el número `n` de bits de entrada de f , y retorna un objeto de la clase `QuantumCircuit` que implementa el algoritmo de Deutsch-Jozsa para `oraculo`. Si `oraculo` tiene $n + r + 1$ qubits, entonces este circuito también debe tener $n + r + 1$ qubits, y debe tener `n` bits clásicos. Considerando la convención anterior, el circuito comienza en el estado $|00^r 0^n\rangle$, y al final mide los qubits $0, \dots, n - 1$ que corresponden a x , almacenando el resultado de medir el qubit i en el bit clásico i .

- Una función `is_constant` con el siguiente encabezado:

```
def is_constant(oraculo: QuantumCircuit, n: int, num_iter: int = 1000) -> bool:
```

Esta función recibe como parámetros un oráculo `oraculo` para una función $f : \{0, 1\}^n \rightarrow \{0, 1\}$ que es constante o balanceada, el número `n` de bits de entrada de f , y el número `num_iter` de veces que se debe ejecutar el circuito. Esta función utiliza un simulador para ejecutar `num_iter` veces el circuito `circuito.deutsch_jozsa(oraculo, n)`, y retorna `True` si f es constante y `False` si f es balanceada. Para la simulación debe utilizar la clase `StatevectorSampler` de `qiskit.primitives`. Si no se indica un valor para `num_iter`, entonces se debe considerar `num_iter = 1000`.

2. [2 puntos] En esta pregunta usted deberá utilizar la librería de la pregunta 1 para determinar si dos funciones son constantes o balanceadas. Dado un string binario u , denotamos por $|u|_1$ el número de símbolos 1 en u . Por ejemplo, si $u = 0110100$, entonces $|u|_1 = 3$. Considere las funciones $f_1, f_2 : \{0, 1\}^7 \rightarrow \{0, 1\}$ definidas de la siguiente forma para cada $x \in \{0, 1\}^7$:

$$\begin{aligned} f_1(x) &= \begin{cases} 1 & \text{si } |x|_1 \leq 3 \\ 0 & \text{en caso contrario} \end{cases} \\ f_2(x) &= 0 \end{aligned}$$

Observe que f_1 es balanceada y que f_2 es constante. Usted debe escribir un programa `pregunta2.py` que realice lo siguiente:

- Construya oráculos para f_1 y f_2 de acuerdo a la convención descrita en la pregunta 1, y los almacene en las variables `oraculo_f1` y `oraculo_f2`, respectivamente. Para construir estos oráculos puede utilizar qubits auxiliares. Tenga en cuenta que estos mismos oráculos se utilizarán en la pregunta 3 en un computador cuántico real, por lo que conviene utilizar pocos qubits auxiliares y pocas compuertas.
- Utilice la función `is_constant` con `num_iter = 1000` para determinar si cada una de estas funciones es constante o balanceada, e imprima el resultado.

El resultado de ejecutar `python pregunta2.py` debe ser exactamente el siguiente:

```
f1 es balanceada
f2 es constante
```

Las variables `oraculo_f1` y `oraculo_f2` se deben definir en el nivel superior del archivo `pregunta2.py`, y el código que utiliza `is_constant` e imprime el resultado debe estar dentro de un bloque de la siguiente forma:

```
if __name__ == "__main__":
    ...
```

De esta forma, los oráculos pueden ser importados desde otro archivo. En la corrección se verificará que `oraculo_f1` y `oraculo_f2` son oráculos para f_1 y f_2 , respectivamente, considerando cada una de las 2^7 entradas posibles.

3. [2 punto] En las preguntas anteriores usted ejecutó el algoritmo de Deutsch-Jozsa en un simulador. En esta pregunta usted deberá ejecutarlo en un computador cuántico de IBM, utilizando su cuenta de IBM Quantum. Parte de esta pregunta consiste en que usted averigüe cómo hacer esto, para lo cual le recomendamos consultar la documentación de IBM Quantum que se encuentra en el sitio <https://quantum.cloud.ibm.com/docs>.

Los computadores cuánticos actuales tienen ruido, vale decir, las compuertas y las mediciones no se realizan de manera perfecta. Por esta razón, el resultado de ejecutar el algoritmo de Deutsch-Jozsa en un computador cuántico real puede ser distinto al resultado que se obtiene en un simulador, y el criterio utilizado en la función `is_constant` para decidir si una función es constante o balanceada podría no ser adecuado. En esta pregunta usted deberá utilizar un criterio que tome en cuenta este ruido.

Además, antes de ser ejecutado en un computador cuántico de IBM, un circuito debe ser transpilado, vale decir, debe ser reescrito utilizando las compuertas y las conexiones entre qubits que tiene dicho computador. En particular, cada compuerta que actúa sobre más de dos qubits se reemplaza por varias compuertas de uno y dos qubits, y pueden ser necesarias compuertas adicionales para respetar las conexiones disponibles entre los qubits. Por esta razón, el circuito transpilado puede ser mucho más grande que el circuito original.

Cada compuerta puede introducir un error, y las compuertas de dos qubits suelen ser especialmente ruidosas. Además, mientras mayor sea la profundidad de un circuito, mayor es la oportunidad de que el ruido afecte su ejecución. Por esta razón, es conveniente que los oráculos que construyó en la pregunta 2 sean lo más pequeños posible. Como referencia, es posible construir un oráculo para f_1 cuyo circuito transpilado tiene alrededor de 500 compuertas de dos qubits en un computador cuántico de la familia Heron de IBM.

El límite de tiempo de uso disponible en IBM Quantum controla cuánto tiempo total de QPU puede utilizar. Este límite no indica si un circuito es suficientemente pequeño para obtener buenos resultados: un circuito puede estar dentro del límite de tiempo y aun así acumular demasiado ruido. Para esta pregunta, usted debe fijarse principalmente en el circuito después de la transpilación y en los resultados obtenidos al ejecutarlo.

Usted debe escribir un programa `pregunta3.py` que ejecute en un computador cuántico de IBM el circuito `circuito_deutsch_jozsa(oraculo, 7)` para `oraculo_f1` y para `oraculo_f2`, con 1000 ejecuciones en cada caso, y que a partir de los resultados obtenidos, y utilizando un criterio que tome en cuenta el ruido, determine si cada una de las funciones f_1 y f_2 es constante o balanceada. Antes de ejecutar cada circuito, debe transpilarlo para el computador cuántico que eligió. Los oráculos se deben importar desde `pregunta2.py`, y la función `circuito_deutsch_jozsa` desde `deutsch_jozsa.py`.

Además, en el documento `pregunta3.pdf` usted debe mostrar que efectivamente ejecutó su programa en un computador cuántico de IBM, y debe explicar el criterio que utilizó. En particular, este documento debe incluir:

- El nombre del computador cuántico que utilizó y la fecha en que ejecutó su programa.
- El identificador (job ID) de cada uno de los trabajos que envió a IBM Quantum para obtener los resultados reportados.
- Un histograma con los resultados obtenidos para cada una de las funciones f_1 y f_2 , indicando explícitamente el número de veces que se obtuvo el string 0^7 .
- El número de compuertas de dos qubits y la profundidad del circuito transpilado para f_1 .
- Una explicación del criterio que utilizó en `pregunta3.py` para decidir si una función es constante o balanceada. En esta explicación debe indicar si este criterio es distinto del utilizado en la función `is_constant` y por qué, y debe justificar, en base a los resultados que obtuvo, por qué su criterio entrega la respuesta correcta para f_1 y f_2 .
- El resultado de ejecutar `pregunta3.py`, el cual debe indicar que f_1 es balanceada y que f_2 es constante.

En la corrección no se ejecutará `pregunta3.py` en un computador cuántico, por lo que la evaluación de esta pregunta se basará en su código y en el documento `pregunta3.pdf`. Tenga en cuenta que el tiempo de uso de los computadores cuánticos de IBM es limitado. Además, los trabajos enviados a IBM Quantum esperan en una cola antes de ser ejecutados. Por esta razón, le recomendamos verificar que su programa funciona correctamente utilizando un simulador antes de ejecutarlo en un computador cuántico, y no dejar esta pregunta para el final.